

Data Modeling for Scale

Phase 2 · Intermediate · Estimated time: 1.5 hours

1. Concept From First Principles

This chapter ties together nearly everything Phase 2 has covered — replication, sharding, consistency, and search — into the practical question that actually kicks off most real system design work: how do you actually shape your data, and which storage technology do you put it in? Getting this decision right early saves enormous rework later; getting it wrong is one of the most common reasons real systems eventually need painful re-architecture.

Traditional relational modeling teaches **normalization**: structure data to minimize redundancy, splitting information into separate tables connected by foreign keys, so that any given fact is stored exactly once (a user's address lives in one row of an addresses table, referenced by ID from anywhere that needs it, rather than copied into every order record). This is the correct default for OLTP (transactional) systems where data integrity and update consistency matter most, and it's what you almost certainly learned first as a backend engineer, and what Eloquent/Sequelize migrations naturally push you toward.

Denormalization deliberately reverses this: it copies data across multiple places to optimize for read speed, at the cost of update complexity and storage. If an order-listing page needs to show the customer's name, address, and the product's title and price, a normalized model requires joining across four tables on every single page load; a denormalized model might store the customer's name and product title directly on the order record itself, at the time the order is placed, so displaying an order is a single-row read with no joins at all — considerably faster, especially once that data is sharded (Chapter 6) across servers where cross-shard joins are expensive or outright impossible.

Backend engineer analogy

This is the real-world version of choosing between an Eloquent `Order::with('customer', 'product')->get()` (normalized, joins at read time) versus simply storing `customer_name` and `product_title` as plain columns directly on the `orders` table at creation time (denormalized, no joins needed, ever, for that specific read). You've likely made small versions of this trade-off already whenever you added a 'snapshot' column to avoid an expensive join — this chapter formalizes when that instinct is right.

Polyglot persistence

A related, equally important idea: large systems rarely use one single database technology for everything. **Polyglot persistence** means deliberately choosing different storage technologies for different data based on that data's actual access patterns — a relational database for transactional order and payment data (needs ACID guarantees, Phase 1 Chapter 5), Elasticsearch for full-text product search (Chapter 13), Redis for session state and caching (Phase 1 Chapter 8), and perhaps a graph database for a social-connections or recommendation feature (naturally relational-in-the-graph-sense data that doesn't fit well into rows and joins). Each technology is chosen because it's genuinely the best fit for that specific slice of data, not because a team standardized on

one database for the entire system.

The key discipline behind good data modeling at scale is **query-driven design**: rather than modeling data the "textbook normalized" way and then figuring out how to query it, you start from the actual, concrete queries your application needs to answer quickly ("get this order with all display data in one read," "get all orders for this user, paginated"), and shape your data model, indexes, and even your storage technology choice around efficiently answering exactly those queries. This is especially central to NoSQL data modeling (document stores like MongoDB, wide-column stores like Cassandra), where the conventional relational advice to "normalize first, optimize later" is often actively counterproductive — these systems are generally designed around denormalized, query-shaped documents from the start.

None of this means normalization is wrong or old-fashioned — it means the choice between normalized and denormalized (and which storage technology) needs to be made deliberately, per piece of data, based on its actual read/write pattern, exactly the way Chapter 3 argued for choosing consistency models per field rather than uniformly. A system's core financial ledger typically stays normalized and relational for correctness; the same system's product-listing read path is very often deliberately denormalized for speed. Treating this as one binary, system-wide decision is a common mistake that interviewers watch for.

2. Real-World Example / Case Study

Flipkart's and Noon's product-listing pages are heavily denormalized by necessity: showing a category page with hundreds of products, each needing title, price, rating, thumbnail, and seller name, would be prohibitively slow if every page load required joining across products, sellers, and reviews tables live — instead, a denormalized, pre-computed "listing view" document (often served through a cache or search index, tying directly back to Chapter 13) is what actually powers the page.

Razorpay's and PhonePe's core ledger, by contrast, stays strictly normalized and relational, because correctness and auditability matter more than raw read speed for financial records — a good illustration that the same company routinely runs both models side by side, deliberately, for different parts of its system.

Careem's and Talabat's tech stacks are classic polyglot-persistence examples: relational databases for trip and payment records, Redis for real-time driver-location caching, Elasticsearch for restaurant/dish search, and potentially a specialized geospatial index for "find drivers near this location" queries — each technology chosen specifically for the access pattern it serves best, not because the team committed to one universal database for the whole platform.

3. Diagram

Normalized (write-optimized, no redundancy):

orders	customers	products
-----	-----	-----
id	id	id
customer_id	----> name	title
product_id	----->	price

Reading one order display = JOIN across 3 tables

Denormalized (read-optimized, some redundancy):

order_display_view

order_id | customer_name | product_title | price
Reading one order display = single-row read, no joins

Polyglot persistence:

Orders/Payments -> Relational DB (ACID)
Product search -> Elasticsearch (inverted index)
Sessions/Cache -> Redis (key-value, fast)
Recommendations -> Graph DB (relationship-heavy queries)

4. Trade-offs Table

Approach	Read speed	Write complexity	Storage
Normalized (relational)	Slower (needs joins)	Simple — update one place	Minimal redundancy
Denormalized	Fast (single-row/document reads)	Complex — must update every copy	More storage used
Single database for everything	Consistent tooling	Simple ops, but suboptimal per-workload fit	N/A
Polyglot persistence	Optimal per data type	More operational surface area to manage	Varies by technology

5. Common Interview Questions

Q1: "Would you denormalize a product-listing page's data model? Why or why not?"

Model approach: Yes, for a high-traffic, read-heavy listing page — justify by explaining that avoiding a multi-table join on every page load meaningfully improves latency at scale, and that the write complexity cost (updating denormalized copies when source data changes) is manageable via async sync pipelines, similar to Chapter 13's search-index sync pattern.

Q2: "What is polyglot persistence, and why not just use one database for the whole system?"

Model approach: Define it as deliberately using different storage technologies matched to each data type's access pattern, and explain that a single database, while operationally simpler, forces every

workload (transactional, search, caching, graph queries) into a tool that's only well-suited to some of them, trading performance and fit for uniformity.

Q3: "How do you keep denormalized copies of data in sync when the source changes?"

Model approach: Describe an async update pipeline — a write to the source of truth triggers an event (via a message queue, Chapter 8) that updates every denormalized copy, accepting eventual consistency between the source and the copies as an acceptable trade-off for the read-speed gain.

Q4: "What is query-driven data modeling, and how does it differ from traditional normalization?"

Model approach: Explain that query-driven modeling starts from the concrete queries the application actually needs answered quickly and shapes the data model around those, whereas traditional normalization starts from minimizing redundancy first and figures out queries afterward — and note this is especially central to NoSQL/document-store design.

6. Practice Exercises

- 1 Take a normalized 3-table schema (users, orders, products) and design a denormalized 'order summary' document optimized for a single-read order history page.
- 2 Propose a polyglot persistence architecture for a ride-hailing app, naming at least three different storage technologies and the specific data/queries each one serves.
- 3 Explain the sync pipeline you'd build to keep a denormalized 'restaurant summary' view up to date when a restaurant's rating changes due to a new review.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Why We Duplicate Data Denormalization Explained System Design"	Direct treatment of why and when denormalization makes sense.
English	"Normalization Vs Denormalization The Art of System Design"	Side-by-side comparison with clear trade-off framing.
Hindi	"What is denormalization in RDBMS in Hindi"	Hindi explanation of denormalization with RDBMS-specific examples.
Hindi	"Normalization vs Denormalization Data Model in Power BI or Power Pivot for Excel in Hindi"	Hindi walkthrough contrasting both approaches with practical examples.

8. Key Takeaways

- Normalization minimizes redundancy and favors write simplicity; denormalization favors read speed at the cost of write complexity.
- Denormalize deliberately for specific, high-traffic read paths, not as a system-wide default.
- Polyglot persistence means choosing different storage technologies matched to each data type's access pattern.
- Query-driven modeling starts from the actual queries you need to answer fast, especially important for NoSQL.

- Denormalized copies need an async sync pipeline (often via message queues) to stay reasonably up to date.
- The same system routinely mixes normalized (ledger/financial) and denormalized (listing/display) models side by side.
- This decision should be made per data type based on its real access pattern, not as one binary, system-wide choice.